

데이터 요약과 시계열 처리

Outline

거래 네 건에서 매장 담당자에게 전달할 범주·월별 매출 요약표를 만든다.

입력과 전달물

- 전: 1월 drink 10·20, 2월 drink 30·food 40.
- 후: 월별 30·70, 범주별 drink 60·food 40.

학습 순서

- 문자열 정리 → 그룹 요약 → 행·열 교차표 → 시간값 변환과 구간 요약.
- 같은 입력의 처리 전후를 비교한 뒤 선택·계산·검산까지 연결한다.

02 · PANDAS

거래 기록 DataFrame 생성

전 · sales 생성에 사용할 값

	category	channel	sales
1	" Drink "	web	10
2	"drink"	web	20
3	"DRINK"	store	30
4	"Food"	store	40

연산 · 분석 노트북 셀

```
import pandas as pd
import numpy as np
sales = pd.DataFrame({
    "category": [" Drink ", "drink",
                 "DRINK", "Food"],
    "channel": ["web", "web", "store",
                "store"],
    "sales": [10, 20, 30, 40],
}, index=[1, 2, 3, 4])
print(sales)
```

후 · 실제 실행 결과

```
category channel  sales
1   Drink      web     10
2  drink      web     20
3  DRINK     store     30
4   Food     store     40
```

거래 한 건이 한 행

- 이름 없는 행 인덱스 1-4가 거래 ID다. category는 범주, channel은 판매 경로, sales는 매출(천 원)이다.
- 공백·대소문자가 다른 세 표기는 모두 음료 범주다. 정리한 뒤 합친다.

표 생성과 전달 목표

- pd는 pandas, np는 NumPy의 별칭이다. {열 이름: 값 목록}으로 세 열을 만들고 index로 행 이름을 붙인다.
- 총매출 100을 범주·경로별로 요약한다. 날짜 열은 시간 처리를 시작할 때 추가한다.

03 · PANDAS

열 전체에 같은 규칙을 적용하는 벡터화 문자열 연산

전 · sales의 category 열

```
category
1  " Drink "
2  "drink"
3  "DRINK"
4  "Food"
```

연산 · 분석 노트북 셀

```
sales["category_clean"] = (
    sales["category"].astype("string")
    .str.strip().str.lower()
)
print(sales["category_clean"])
```

후 · 실제 실행 결과

```
1  drink
2  drink
3  drink
4  food
Name: category_clean, dtype:
string
```

연산 순서와 각 줄의 역할

- `astype("string")`은 결측을 보존하는 문자열 자료형으로 바꾼다. `str`은 열 전체에 문자열 연산을 적용하는 접근자다.
- `strip()`은 양끝 공백을, `lower()`는 영문 대소문자 차이를 없앤다. " Drink " → "Drink" → "drink"다.

변한 값과 유지된 대응

- 음료 세 행이 같은 키 `drink`를 갖는다. 매출 10·20·30과 거래 ID 1·2·3은 유지된다.
- 원본 `category` 열을 남기고 `category_clean` 열을 추가한다. 반복문 없이 모든 행에 같은 규칙을 적용한다.

groupby의 분할·적용·결합 구조

전 · 교재 그림의 Key·Data

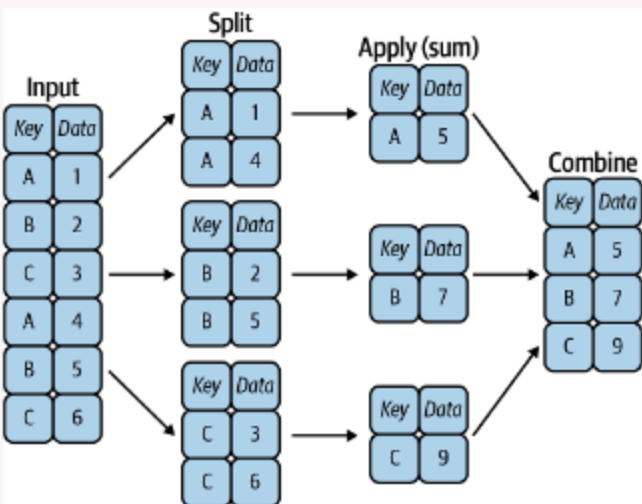
	Key	Data
0	A	1
1	B	2
2	C	3
3	A	4
4	B	5
5	C	6

연산 · 분석 노트북 셀

```
example = pd.DataFrame({
    "Key": list("ABCABC"),
    "Data": [1, 2, 3, 4, 5, 6]
})
print(example.groupby("Key")
      ["Data"].sum())
```

후 · 실제 실행 결과

```
Key
A    5
B    7
C    9
Name: Data, dtype: int64
```



왼쪽 원본 → 같은 키의 묶음 → 각 합계 → 오른쪽 결과. 교재 Figure 20-1.

분할 → 적용 → 결합

- 그룹은 키가 같은 원본 행의 묶음이다. A는 행 0·3의 값 [1,4], B는 [2,5], C는 [3,6]을 받는다.
- 각 묶음에 sum을 적용한 5·7·9를 키별 한 행으로 결합한다. groupby만 호출하면 아직 합계 표가 아니다.

05 · PANDAS

그룹별 합계·건수·평균

전 · sales의 관련 두 열

	category_clean	sales
1	drink	10
2	drink	20
3	drink	30
4	food	40

연산 · 분석 노트북 셀

```
summary =
sales.groupby("category_clean").agg(
    n=("sales", "size"),
    total=("sales", "sum"),
    avg=("sales", "mean")
)
print(summary)
```

후 · 실제 실행 결과

	n	total	avg
category_clean			
drink	3	60	20.0
food	1	40	40.0

집계의 입력과 결과

- drink 그룹은 ID 1·2·3의 세 행이다. 건수 3, 합계 10+20+30=60, 평균 60/3=20이다. food는 한 행이다.
- agg(결과열=(입력열, 함수))다. n·total·avg는 새 열 이름이다. 키가 결과 인덱스가 되고 4행이 2행으로 줄어든다.

결측과 분모

- size는 전체 행 수, count는 유효값 수다. [10, 결측, 30]이면 size=3, count=2, mean=20이다.
- 이 입력의 매출·범주에는 결측이 없다. 그룹 키의 결측은 기본 제외되므로 실제 자료에서는 먼저 검사한다.

06 · PANDAS

집계·변환·필터의 출력 행 수

전 · sales의 관련 열과 그룹

	category_clean	sales
1	drink	10
2	drink	20
3	drink	30
4	food	40

drink 그룹: ID 1·2·3 → 합 60
food 그룹: ID 4 → 합 40

연산 · 분석 노트북 셀

```
group_total = sales.groupby(
    "category_clean")["sales"].transform("sum")
groups = sales.groupby("category_clean")
kept = groups.filter(lambda g: len(g) >= 3)
print(group_total)
print("남은 ID:", kept.index.tolist())
```

후 · group_total / 남은 ID

1	60
2	60
3	60
4	40

Name: sales, dtype: int64
남은 ID: [1, 2, 3]

합계를 원래 행에 대응시키는 transform

- drink의 합계 60을 ID 1·2·3 각각에, food의 합계 40을 ID 4에 붙인다. 결과는 ID를 유지한 4행이다.
- group_total은 그룹당 한 행의 표가 아니다. 예를 들어 ID 1의 그룹 내 비율은 10/60, ID 2는 20/60이다.

그룹 전체를 선택하는 filter

- lambda g는 이름 없는 함수다. g에 drink의 원본 세 행이 들어오면 len(g)=3 → 참, food의 한 행이면 1 → 거짓이다.
- 참인 그룹의 원본 행만 남긴다. 집계는 2행, transform은 4행, 이 filter는 ID 1·2·3의 3행이다.

07 · PANDAS

두 범주를 행과 열로 배치하는 피벗 테이블

전 · sales의 교차 요약 입력

	category_clean	channel	sales
1	drink	web	10
2	drink	web	20
3	drink	store	30
4	food	store	40

연산 · 분석 노트북 셀

```
matrix = sales.pivot_table(
    values="sales",
    index="category_clean",
    columns="channel",
    aggfunc="sum",
    fill_value=0,
)
print(matrix)
```

후 · 실제 실행 결과

channel	store	web
category_clean		
drink	30	30
food	40	0

행·열·셀의 대응

- category_clean은 범주, channel은 판매 경로, sales는 매출이다. index는 결과 행, columns는 결과 열, values는 요약할 값을 지정한다.
- drink×web에는 ID 1·2가 들어가므로 10+20=30이다. drink×store는 30, food×store는 40이다.

집계 방식과 빈 조합

- 기본 집계는 평균이다. drink×web의 기본값 15와 합계 30은 다르므로 aggfunc="sum"을 명시한다.
- food×web 거래는 없다. 전체 거래가 수록됐다는 가정으로 fill_value=0을 사용한다. 기록 누락이면 0으로 단정할 수 없다.

08 · PANDAS

문자열 분해와 결측을 포함한 조건

전 · 별도 상품 태그 세 값

```
0 "Latte|HOT"
1 "hotdog|cold"
2 결측
```

연산 · 분석 노트북 셀

```
tags = pd.Series(
    ["Latte|HOT", "hotdog|cold", None],
    dtype="string")
parsed = tags.str.split("|", expand=True)
parsed.columns = ["item", "tag"]
print(parsed.to_string(index=False))
print(tags.str.contains("hot", case=False,
na=False).tolist())
print((parsed["tag"].str.lower() ==
"hot").fillna(False).tolist())
```

후 · 실제 실행 결과

```
item tag
Latte HOT
hotdog cold
      <NA> <NA>
[True, True, False]
[True, False, False]
```

분해 결과의 두 열

- split("|", expand=True)는 구분자 양쪽을 별도 열로 만든다. 기본 False면 한 셀에 리스트를 넣는다.
- 이 입력은 구분자가 하나다. columns로 두 열에 item·tag 이름을 붙이며 결측은 두 열 모두 <NA>로 남는다.

부분 일치와 정확한 태그

- 출력 첫 목록은 contains의 부분 일치다. case=False는 대소문자 무시, na=False는 결측을 거짓으로 정한다.
- hotdog도 부분 일치는 참이다. 두 번째 목록은 태그만 소문자로 바꿔 hot과 비교하고 결측을 거짓으로 채운 결과다.

09 · PANDAS

시점·기간·간격과 시간 격자

전 · 시점·기간·간격의 입력

시점: 2026-02-01 09:30
기간: 2026-02 한 달
간격: 2일

격자: 03-01부터 하루 간격 3개

연산 · 분석 노트북 셀

```
stamp = pd.Timestamp("2026-02-01 09:30")
period = pd.Period("2026-02", freq="M")
delta = pd.Timedelta(days=2)
days = pd.date_range("2026-03-01",
                      periods=3, freq="D")
print(stamp + delta)
print(period)
print(days.strftime("%m-%d").tolist())
```

후 · 실제 실행 결과

```
2026-02-03 09:30:00
2026-02
['03-01', '03-02', '03-03']
```

시간값의 의미

- Timestamp는 한 시점, Period는 한 달 같은 기간, Timedelta는 시간 간격이다. 시점에 이들을 더해도 시각 09:30은 유지된다.
- freq="M"은 여기서 월 기간을 뜻한다. 같은 문자열처럼 보여도 날짜 문자열과 계산 가능한 시간 객체는 다르다.

예상 시점의 목록

- date_range는 시작일·개수(periods)·간격(freq)으로 시점 목록을 만든다. D는 하루 간격이다.
- strftime은 날짜의 표시 형식만 바꾼다. %m은 월, %d는 일이다. 빈 날짜가 기록 누락인지 거래 미발생인지는 별도 판단한다.

날짜 변환과 실패·결측 검사

전 · 별도 날짜 진단 입력

```
0 "2026-02-28"
1 "2026-02-30"
2 원래 결측
```

연산 · 분석 노트북 셀

```
sales["occurred_at"] = [
    "2026-01-03", "2026-01-10",
    "2026-02-01", "2026-02-14",
]
sales["occurred_at"] =
pd.to_datetime(sales["occurred_at"])
assert sales["occurred_at"].notna().all()
probe = pd.Series(["2026-02-28", "2026-02-30",
None])
checked = pd.to_datetime(probe, errors="coerce")
print(checked)
print("결측:", checked.isna().sum())
```

후 · 실제 실행 결과

```
0    2026-02-28
1           NaT
2           NaT
dtype: datetime64[ns]
결측: 2
```

기본 변환과 조사용 변환

- 시간 처리를 시작하며 occurred_at 열을 추가한다. 목록의 네 날짜는 ID 1-4 순서다. to_datetime으로 시점으로 바꾸고 notna().all()로 결측 없음을 검사한다.
- 별도 probe에서는 존재하지 않는 2월 30일과 원래 결측이 모두 NaT가 된다. NaT는 시간 결측이다.

실패와 결측의 구분

- 기본 errors="raise"는 잘못된 날짜에서 중단한다. coerce는 변환 실패를 NaT로 남겨 원자료를 조사하게 한다.
- isna().sum()은 결측 수 2를 센다. raise도 원래 결측은 허용할 수 있으므로 변환 성공과 필수값 검사는 별개다.

11 · PANDAS

시간 범위 선택 기준인 DatetimeIndex

전 · sales의 시점 열 · 2026년

	occurred_at	sales
1	2026-01-03	10
2	2026-01-10	20
3	2026-02-01	30
4	2026-02-14	40

연산 · 분석 노트북 셀

```
ts =
sales.set_index("occurred_at").sort_index()
print(type(ts.index).__name__)
print(ts.loc["2026-02", ["sales"]])
```

후 · 실제 실행 결과

```
DatetimeIndex
sales
occurred_at
2026-02-01      30
2026-02-14      40
```

행 이름을 시점으로 전환

- set_index는 occurred_at 열을 행 선택 기준으로 옮긴다. sort_index는 그 시점을 오름차순 정렬한다.
- ts의 인덱스는 DatetimeIndex다. sales 원본의 ID 인덱스와 별개인 새 표를 만든다.

달 이름으로 구간 선택

- loc[행 조건, 열 목록]이다. "2026-02"는 2월의 모든 시점을, ["sales"]는 매출 열을 선택한다.
- 전의 ID 3·4가 후의 2월 두 행이다. 매출 30+40=70이며 1월 두 행은 결과에서 제외된다.

12 · PANDAS

같은 시간 구간의 관측을 요약하는 resample

전 · ts의 시점별 매출 · 2026년

occurred_at	sales
2026-01-03	10
2026-01-10	20
2026-02-01	30
2026-02-14	40

연산 · 분석 노트북 셀

```
monthly = ts["sales"].resample("MS").sum()
print(monthly)
```

후 · 실제 실행 결과

```
occurred_at
2026-01-01    30
2026-02-01    70
Freq: MS, Name: sales, dtype:
int64
```

시점을 월 구간으로 묶기

- resample("MS")는 월 시작을 기준으로 나눈다. MS는 월 시작이다. 1월 구간은 01-01 이상·02-01 미만이다.
- 1월의 10+20=30, 2월의 30+40=70이다. sum()을 적용해 관측 4행을 월별 2행으로 요약한다.

결과의 날짜 읽기

- 결과 01-01·02-01은 각 구간의 라벨이다. 실제로 그 날짜에 발생한 거래만 더한 값이 아니다.
- 총합 30+70=100은 원자료 합계와 같다. 시간 구간과 집계 함수를 함께 정해야 한다.

resample의 구간 집계와 asfreq의 격자 선택

전 · daily · 이름 없는 Series

```
2026-03-01    10
2026-03-02    20
2026-03-03    30
```

연산 · 분석 노트북 셀

```
daily = pd.Series([10, 20, 30],
    index=pd.date_range("2026-03-01",
periods=3))
print("구간 합:",
daily.resample("2D").sum().tolist())
print("격자 값:",
daily.asfreq("2D").tolist())
```

후 · 실제 실행 결과

```
구간 합: [30, 30]
격자 값: [10, 30]
```

같은 입력의 두 결과

- 2D는 이틀 간격이다. resample은 1·2일을 묶어 10+20=30, 3일 구간은 30을 낸다. 결과 날짜는 1·3일이다.
- asfreq는 1·3일 격자에 있는 기존 값 10·30을 고른다. 2일의 20을 1일 값에 합치지 않는다.

선택 기준

- 시간 구간의 통계가 필요하다면 resample, 다른 시간 격자에 맞춘 기존 관측값이 필요하다면 asfreq를 쓴다.
- 이 장부터 빈 칸 채우기까지는 별도 일별 관측 예제다. 거래 네 건을 월별로 요약하는 앞 단계와 구분한다.

14 · PANDAS

빈 시간 칸의 채우기 방향과 가정

전 · sparse · 이름 없는 Series

```
2026-03-01    10
2026-03-03    30
```

연산 · 분석 노트북 셀

```
sparse = daily.loc[["2026-03-01", "2026-03-03"]]
grid = sparse.asfreq("D")
print(pd.DataFrame({
    "raw": grid,
    "ffill": grid.ffmpeg(),
    "bfill": grid.bfill(),
}))
```

후 · 실제 실행 결과

raw	ffill	bfill
2026-03-01	10.0	10.0
2026-03-02	NaN	10.0
2026-03-03	30.0	30.0

격자 생성과 값 채우기

- asfreq("D")는 2일 행을 새로 만들고 관측이 없으므로 NaN을 둔다. ffill은 앞의 10, bfill은 뒤의 30으로 그 칸을 채운다.
- 1·3일의 실제 관측값은 변하지 않는다. 오른쪽 세 열은 같은 결측에 적용할 대안이다.

각 대안이 추가하는 가정

- ffill은 직전 상태 유지, bfill은 다음 값을 앞당겨 사용한다는 가정이 다. 미래값을 쓸 수 없는 예측에는 bfill이 부적절하다.
- fillna(0)은 0이라는 별도 가정이다. 거래 미발생과 기록 누락을 구분한 뒤 방법을 선택한다.

15 · PANDAS

직전 값 비교와 두 관측 이동 평균

전 · 공통 거래로 만든 monthly

occurred_at	sales
2026-01-01	30
2026-02-01	70

연산 · 분석 노트북 셀

```
change = pd.DataFrame({"sales":
monthly})
change["previous"] =
monthly.shift(1)
change["change"] = monthly -
change["previous"]
change["rolling_2"] =
monthly.rolling(2).mean()
print(change.rename(columns={
"previous":"prev",
"rolling_2":"avg2"
})))
```

후 · 실제 실행 결과

sales	prev	change	avg2
occurred_at			
2026-01-01	30	NaN	NaN
2026-02-01	70	30.0	40.0

직전 값을 같은 행으로 가져오기

- 이제 공통 거래의 월별 표로 돌아온다. shift(1)은 인덱스를 유지하고 값을 한 행 아래로 옮긴다. shift(-1)은 다음 값을 가져온다.
- 2월의 prev=30, 변화량=70-30=40이다. 첫 달은 직전 값이 없어 NaN이다. rename은 출력의 previous·rolling_2 이름을 prev·avg2로 줄인다.

현재와 직전 관측의 평균

- rolling(2).mean()은 두 행 창의 평균이다. 기본 min_periods=2·center=False로 현재와 직전 값이 모두 필요하다.
- 2월 avg2=(30+70)/2=50이다. 첫 NaN을 0으로 바꾸지 않는다. 이 표는 매달 한 행이지만 불규칙한 두 거래는 두 달이 아니다.

16 · PANDAS

문자열 식으로 행 선택과 열 계산

전 · sales의 관련 두 열

	sales	channel
1	10	web
2	20	web
3	30	store
4	40	store

연산 · 분석 노트북 셀

```
threshold = 15
selected = sales.query('sales >= @threshold
and channel == "web"')
with_tax = sales.eval("tax = sales * 0.1")
print("선택 ID:", selected.index.tolist())
print("세금:", with_tax["tax"].tolist())
print(np.allclose(with_tax["tax"],
sales["sales"] * 0.1))
```

후 · 실제 실행 결과

```
선택 ID: [2]
세금: [1.0, 2.0, 3.0, 4.0]
True
```

query: 조건으로 원본 행 선택

- 식 안의 sales·channel은 열 이름이다. @threshold는 식 밖의 Python 변수 15를 참조한다.
- 매출 15 이상과 web 조건을 모두 만족하는 ID 2만 남는다. 원본 네 행은 수정하지 않는다.

eval: 식으로 새 열 계산

- tax는 설명용 10% 세금 열이다. eval은 각 행의 매출에 0.1을 곱한 새 표를 반환한다. 원본 sales에는 tax가 추가되지 않는다.
- np.allclose는 일반 열 연산과 값이 허용오차 안에서 같은지 검사한다. 인덱스·열 이름까지 비교하는 검사는 아니다.

17 · PANDAS

큰 중간 배열에서의 eval·query 선택 조건

전 · sales의 비교 대상

	sales	channel
1	10	web
2	20	web
3	30	store
4	40	store

연산 · 분석 노트북 셀

```
tmp1 = sales["sales"] >= 15
tmp2 = sales["channel"] == "web"
mask = tmp1 & tmp2
print(tmp1.tolist())
print(tmp2.tolist())
print(mask.tolist())
print("bytes:", tmp1.values.nbytes,
      tmp2.values.nbytes, mask.values.nbytes)
```

후 · 실제 실행 결과

```
[False, True, True, True]
[True, True, False, False]
[False, True, False, False]
bytes: 4 4 4
```

입력 네 행 → 논리 배열 세 개

- 첫 조건은 [거짓,참,참,참], 둘째는 [참,참,거짓,거짓]이다. &는 같은 ID의 조건을 결합해 ID 2만 참으로 만든다.
- bool 값은 여기서 각 1바이트다. 네 행×세 배열=12바이트, 1천만 행이면 배열 데이터만 30,000,000바이트다.

비용을 측정한 뒤 대안 선택

- nbytes는 배열 데이터량이며 인덱스·엔진 버퍼를 제외한다. query의 실제 절감량이나 최고 메모리를 뜻하지 않는다.
- NumExpr는 지원하는 수치식을 묶음 계산하는 엔진이다. 작은 표는 일반식으로 충분하고 큰 표는 동치성·시간·메모리를 실측한다.

18 · PANDAS

정리·시간화·요약 순서를 기록하는 통합 파이프라인

전 · sales의 관련 열

	occurred_at	category_clean	sales
1	2026-01-03	drink	10
2	2026-01-10	drink	20
3	2026-02-01	drink	30
4	2026-02-14	food	40

연산 · 분석 노트북 셀

```
prepared = sales.assign(
    category=sales["category_clean"]
).set_index("occurred_at").sort_index()
by_month = prepared.groupby([
    pd.Grouper(freq="MS"), "category"
])["sales"].sum()
plot_data =
by_month.unstack(fill_value=0)
print(plot_data)
```

후 · 실제 실행 결과

category	drink	food
occurred_at		
2026-01-01	30	0
2026-02-01	30	40

개별 기능 → 전달할 표 조립

- 범주와 날짜는 앞에서 정리했다. assign은 category_clean 값을 category에 넣은 새 표를 만들고 set_index·sort_index는 시간 그룹 기준을 만든다.
- Grouper(freq="MS")는 월별 묶기 규칙이다. 월·범주로 합치면 (1월,drink)=30, (2월,drink)=30, (2월,food)=40이다.

묶인 결과를 두 축에 배치

- unstack은 마지막 키 category를 열로 옮긴다. 1월 drink의 합계 30이 1월 행·drink 열의 같은 값으로 이동한다.
- 전체 거래가 있으므로 1월 food 미발생 조합을 0으로 둔다. 앞의 범주×경로 표와 달리 이 표는 월×범주 2×2다.

질문에 따른 동일 요약값의 시각화

전 · 같은 네 거래의 요약값

```
monthly: [30, 70]
summary["total"]: [60, 40]
```

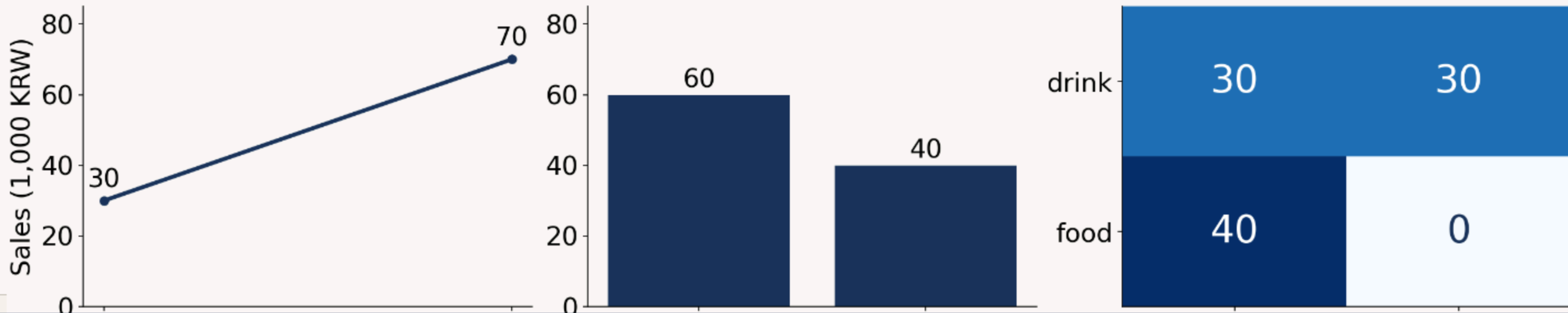
```
matrix
channel      store  web
category_clean
drink         30   30
food          40    0
```

연산 · 분석 노트북 셀

```
# 앞에서 만든 세 요약표
print("월별:", monthly.tolist())
print("범주별:", summary["total"].tolist())
print("교차 셀:", matrix.values.tolist())
```

후 · 실제 실행 결과

```
월별: [30, 70]
범주별: [60, 40]
교차 셀: [[30, 30], [40, 0]]
```



값을 질문에 맞는 도형으로 옮기기

- 선의 두 점은 30→70으로 시간 변화를, 막대 높이 60·40은 범주 크기를 나타낸다. 색상표의 숫자는 store·web 교차 셀이다.
- 숫자는 그대로 유지하고 표현만 바꾼다. 월×범주 plot_data의 행 합은 30·70, 열 합은 60·40이다.

작은 불변량을 이용한 전체 결과 검산

전 · 검산할 plot_data

occurred_at	drink	food
2026-01-01	30	0
2026-02-01	30	40

열 축 이름: category

총합 100, 2x2 표

반례: 첫 행 [30,0] → [29,1]

연산 · 분석 노트북 셀

```
expected = [[30, 0], [30, 40]]
assert len(sales) == 4 and sales.index.is_unique
assert prepared.index.is_monotonic_increasing
assert not prepared.index.hasnans
assert plot_data.shape == (2, 2)
assert plot_data.values.sum() == sales["sales"].sum()
== 100
assert plot_data.values.tolist() == expected
bad = plot_data.copy()
bad.iloc[0] = [29, 1]
print("정상: 모든 검사 통과")
print("반례 합계:", bad.values.sum() == 100)
print("반례 셀:", bad.values.tolist() == expected)
```

후 · 실제 실행 결과

정상: 모든 검사 통과

반례 합계: True

반례 셀: False

구조·시간·값을 나누어 검사

- assert는 조건이 거짓이면 중단한다. is_unique는 ID 중복 없음, is_monotonic_increasing은 오름차순, hasnans는 결측 유무다.
- shape는 행·열 수다. values.tolist()로 행 순서의 값 목록을 만들고 기대하는 네 셀과 비교한다.

합계 검사의 한계

- 반례는 1월 drink에서 1을 빼고 food에 1을 더했다. iloc[0]은 첫 행을 위치로 선택한다. 총합 100은 그대로다.
- 합계 검사는 참이지만 셀 검사는 거짓이다. 실제 업무에서는 행·열 이름도 확인하고 기대값은 별도로 계산해야 한다.

데이터 요약과 시계열 처리

Summary

입력의 각 행이 어디로 이동하고 어떻게 합쳐지는지 추적해 결과를 검산한다.

네 거래에서 요약표까지

- drink 10·20·30 → 합계 60. transform은 60을 원래 세 ID에 각각 대응시킨다.
- 1월 10·20 → 30, 2월 30·40 → 70. 직전값 비교는 40, 두 달 평균은 50이다.

선택과 검산

- 문자열·날짜를 정리하고 목적에 따라 집계·격자 선택·채우기·행 선택·열 계산을 구분한다.
- 총합 100뿐 아니라 결과 셀 [30,0]·[30,40]도 확인한 후 시각화 입력으로 전달한다.